NEMOX AI | RICO-7

Three Solutions: Complete Source Code Listing

All non-directory files from NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION.zip, reproduced in full and ordered by repository path. This is the source as delivered in the implementation archive, not an assertion of production completeness.

File count: 39

01. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/.github/workflows/tests.yml

```
name: Python API Tests
on: [push, pull_request]
jobs:
  tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        solution: [01_user_intent_inspector, 02_recursive_feedback_engine,
03_understanding_gap_detector]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r ${{ matrix.solution }}/requirements.txt
      - run: cd ${{ matrix.solution }} && PYTHONPATH=.:../shared pytest -q
```

02. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY shared /app
COPY 01_user_intent_inspector/backend /app/backend
COPY 01_user_intent_inspector/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
ENV PYTHONPATH=/app
CMD ["uvicorn","backend.main:app","--host","0.0.0.0","--port","8011"]
```

03. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/README.md

```
# User Intent & Context Inspector

Independent developer tool using shared field contracts.

Backend: `PYTHONPATH=.:../shared uvicorn backend.main:app --port 8011` from this project directory
after `pip install -r requirements.txt`.
Frontend: `cd frontend && npm install && npm run dev` (port 9011).
Tests: `PYTHONPATH=.:../shared pytest -q`.
Swagger: http://localhost:8011/docs

Demo data is synthetic. SQLite DB is created on first launch.
```

04. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/backend/__init__.py

```
[EMPTY FILE]
```

05. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/backend/main.py

```
from fastapi import FastAPI,HTTPException
from pydantic import BaseModel,Field
from fieldcore import uid
from store import Store
import os,re
app=FastAPI(title='User Intent Inspector')
store=Store(os.getenv('DB_PATH','intent.sqlite'))
class Request(BaseModel):
 request:str=Field(min_length=1)
 customer_id:str='demo-customer'
class History(BaseModel):
 shipment_id:str
 customer_id:str='demo-customer'
 delivery_date:str
 carrier:str
@app.post('/api/history')
def add_history(h:History):return {'id':store.save('history',h.model_dump())}
@app.post('/api/analyze')
def analyze(req:Request):
 text=req.request;ref=bool(re.search(r'last week|same shipment|as usual',text,re.I))
 match=re.search(r'(\d+|one|two|three|four|five) days? earlier',text,re.I)
 delta=-(int(match.group(1)) if match.group(1).isdigit() else
```

```
{'one':1,'two':2,'three':3,'four':4,'five':5}[match.group(1).lower()]) if match else None
 candidates=[x for x in store.list('history') if x['customer_id']==req.customer_id] if ref else []
 gaps=[]
 if ref and
len(candidates)!=1:gaps.append({'code':'HISTORICAL_REFERENCE','reason':f'{len(candidates)} candidate
 shipments','status':'unresolved'})
 if delta is not None and len(candidates)!=1:gaps.append({'code':'BASELINE_DATE','reason':'relative
date requires resolved shipment','status':'unresolved'})
 requirements=[{'kind':'delivery_delta_days','value':delta,'source_ref':'request:relative-date'}] if
 delta is not None else []
 result={'trace_id':uid(),'original_request':text,'fields':{'F02':{'request':text},'F04':{'historica
l_reference':ref},'F05':{'requirements':requirements}},'candidate_shipments':candidates,'unresolved'
:gaps,'relations':[{'predicate':'REFERENCES_SHIPMENT','status':'resolved' if len(candidates)==1 else
 'unresolved'}] if ref else []}
 store.save('trace',result,result['trace_id']);store.event('INTERPRETATION_CREATED',result['trace_id
'],result)
 return result
@app.get('/api/traces/{trace_id}')
def trace(trace_id:str):
 x=store.get(trace_id)
 if x is None:raise HTTPException(404,'trace not found')
 return x
@app.get('/api/history')
def history():return store.list('history')
```

06. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/frontend/index.html

```
<html><head><meta name="viewport" content="width=device-width,initial-scale=1" /></head><body><div
id="root"></div><script type="module" src="/src.tsx"></script></body></html>
```

07. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/frontend/package.json

```
{
  "name": "01_user_intent_inspector",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "tsc && vite build",
    "test": "vitest run"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "reactflow": "^11.11.4"
  },
  "devDependencies": {
    "typescript": "^5.6.3",
    "vite": "^5.4.10",
    "vitest": "^2.1.4",
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1"
  }
}
```

08. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/frontend/src.tsx

```
import React, {useState} from 'react';
import {createRoot} from 'react-dom/client';
import ReactFlow, {Background,Controls} from 'reactflow';
import 'reactflow/dist/style.css';
const title="User Intent & Context Inspector";
function App() {
 const [input,setInput]=useState('Same shipment as last week, but two days earlier.');
 const [result,setResult]=useState<any>(null); const [error,setError]=useState('');
 const endpoint="/api/analyze";
 async function submit() {try{const body=endpoint==='/api/decisions'?{request:input,carrier:'Carrier
 A'}:{request:input};const r=await fetch(endpoint,{method:'POST',headers:{'Content-Type':'applicatio
n/json'},body:JSON.stringify(body)});const x=await r.json();if(!r.ok)throw
Error(JSON.stringify(x));setResult(x);setError('')}catch(e){setError(String(e))}}
 const nodes=result?[{id:'1',position:{x:0,y:20},data:{label:'User
request'}},{id:'2',position:{x:240,y:20},data:{label:'Context
```

```
field'}},{id:'3',position:{x:480,y:20},data:{label:'Developer inspection'}}]:[];
 const edges=[{id:'e1',source:'1',target:'2'},{id:'e2',source:'2',target:'3'}];
 return <main style={{fontFamily:'system-ui',maxWidth:1000,margin:'32px
auto',padding:20}}><h1>{title}</h1><p>Independent field-based developer inspection tool. Synthetic
data only.</p><textarea value={input} onChange={e=>setInput(e.target.value)}
style={{width:'100%',height:100}}/><p><button onClick={submit}>Process
request</button></p>{error&&<p role="alert">{error}</p>}{result&&<><section
style={{height:180,border:'1px solid #ddd'}}><ReactFlow nodes={nodes} edges={edges}
fitView><Background/><Controls/></ReactFlow></section><h2>Inspection result</h2><pre style={{whiteSp
ace:'pre-wrap',background:'#f4f5f7',padding:18}}>{JSON.stringify(result,null,2)}</pre></>}</main>
}
createRoot(document.getElementById('root')!).render(<App/>);
```

09. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/frontend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "lib": [
      "ES2020",
      "DOM"
    ]
  },
  "include": [
    "src.tsx"
  ]
}
```

10. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/frontend/vite.config.ts

```
import { defineConfig } from 'vite';
export default defineConfig({server:{port:9011,proxy:{'/api':'http://localhost:8011'}}});
```

11. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/requirements.txt

```
fastapi>=0.110,<1
uvicorn>=0.29,<1
pydantic>=2.6,<3
pytest>=8,<10
httpx>=0.27,<1
```

12. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/01_user_intent_inspector/tests/test_api.py

```
from fastapi.testclient import TestClient
from backend.main import app
client=TestClient(app)
def test_intent():
 r=client.post('/api/analyze',json={'request':'Same shipment as last week, but two days
earlier.'});assert r.status_code==200
 assert r.json()['unresolved'];assert r.json()['fields']['F05']['requirements'][0]['value']==-2
```

13. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY shared /app
COPY 02_recursive_feedback_engine/backend /app/backend
COPY 02_recursive_feedback_engine/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
ENV PYTHONPATH=/app
```

```
CMD ["uvicorn","backend.main:app","--host","0.0.0.0","--port","8012"]
```

14. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/README.md

```
# Recursive Feedback & Decision Trace Engine

Independent developer tool using shared field contracts.

Backend: `PYTHONPATH=.:../shared uvicorn backend.main:app --port 8012` from this project directory
after `pip install -r requirements.txt`.
Frontend: `cd frontend && npm install && npm run dev` (port 9012).
Tests: `PYTHONPATH=.:../shared pytest -q`.
Swagger: http://localhost:8012/docs

Demo data is synthetic. SQLite DB is created on first launch.
```

15. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/backend/__init__.py

```
[EMPTY FILE]
```

16. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/backend/main.py

```
from fastapi import FastAPI,HTTPException
from pydantic import BaseModel,Field
from fieldcore import uid
from store import Store
import os
app=FastAPI(title='Recursive Feedback Engine')
store=Store(os.getenv('DB_PATH','feedback.sqlite'))
class DecisionInput(BaseModel):
 request:str=Field(min_length=1)
 carrier:str
 customer_id:str='demo-customer'
class FeedbackInput(BaseModel):
 decision_id:str
 corrected_carrier:str
 reason:str|None=None
class Confirmation(BaseModel):
 correction_id:str
 confirmed:bool
class Query(BaseModel):
 customer_id:str='demo-customer'
 request:str
@app.post('/api/decisions')
def decision(d:DecisionInput):
 x={'decision_id':uid(),**d.model_dump(),'context_snapshot':{'customer_id':d.customer_id},'status':'
proposed'}

store.save('decision',x,x['decision_id']);store.event('DECISION_CAPTURED',x['decision_id'],x);return
 x
@app.post('/api/feedback')
def feedback(f:FeedbackInput):
 d=store.get(f.decision_id)
 if not d or 'decision_id' not in d:raise HTTPException(404,'decision not found')
 x={'correction_id':uid(),**f.model_dump(),'old_carrier':d['carrier'],'customer_id':d['customer_id']
,'status':'proposed'}
 store.save('correction',x,x['correction_id']);store.event('CORRECTION_PROPOSED',f.decision_id,x);re
turn x
@app.post('/api/confirm')
def confirm(c:Confirmation):
 x=store.get(c.correction_id)
 if not x or 'correction_id' not in x:raise HTTPException(404,'correction not found')
 if x['status']!='proposed':raise HTTPException(409,'already reviewed')
 x={**x,'status':'confirmed' if c.confirmed else 'rejected'}
 store.save('review',x);store.event('CORRECTION_REVIEWED',x['decision_id'],x);return x
@app.post('/api/context/retrieve')
def retrieve(q:Query):
 confirmed=[x for x in store.list('review') if x['status']=='confirmed' and
x['customer_id']==q.customer_id]
 selected=confirmed[-1:] if confirmed else []
 return {'request':q.request,'customer_id':q.customer_id,'retrieved_relations':[{'carrier':x['correc
ted_carrier'],'source_correction_id':x['correction_id']} for x in selected]}
```

```
@app.post('/api/compare')
def compare(q:Query):
 ctx=retrieve(q);carrier=ctx['retrieved_relations'][0]['carrier'] if ctx['retrieved_relations'] else
 'Carrier A'
 return {'input':q.request,'without_feedback':{'carrier':'Carrier A'},'with_feedback':{'carrier':car
rier},'retrieval_manifest':ctx['retrieved_relations'],'changed':carrier!='Carrier A'}
@app.get('/api/timeline')
def timeline():return {'decisions':store.list('decision'),'corrections':store.list('correction'),'re
views':store.list('review')}
```

17. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/frontend/index.html

```
<html><head><meta name="viewport" content="width=device-width,initial-scale=1" /></head><body><div
id="root"></div><script type="module" src="/src.tsx"></script></body></html>
```

18. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/frontend/package.json

```
{
  "name": "02_recursive_feedback_engine",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "tsc && vite build",
    "test": "vitest run"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "reactflow": "^11.11.4"
  },
  "devDependencies": {
    "typescript": "^5.6.3",
    "vite": "^5.4.10",
    "vitest": "^2.1.4",
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1"
  }
}
```

19. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/frontend/src.tsx

```
import React, {useState} from 'react';
import {createRoot} from 'react-dom/client';
import ReactFlow, {Background,Controls} from 'reactflow';
import 'reactflow/dist/style.css';
const title="Recursive Feedback & Decision Trace Engine";
function App() {
 const [input,setInput]=useState('Same shipment as last week, but two days earlier.');
 const [result,setResult]=useState<any>(null); const [error,setError]=useState('');
 const endpoint="/api/decisions";
 async function submit() {try{const body=endpoint==='/api/decisions'?{request:input,carrier:'Carrier
 A'}:{request:input};const r=await fetch(endpoint,{method:'POST',headers:{'Content-Type':'applicatio
n/json'},body:JSON.stringify(body)});const x=await r.json();if(!r.ok)throw
Error(JSON.stringify(x));setResult(x);setError('')}catch(e){setError(String(e))}}
 const nodes=result?[{id:'1',position:{x:0,y:20},data:{label:'User
request'}},{id:'2',position:{x:240,y:20},data:{label:'Context
field'}},{id:'3',position:{x:480,y:20},data:{label:'Developer inspection'}}]:[];
 const edges=[{id:'e1',source:'1',target:'2'},{id:'e2',source:'2',target:'3'}];
 return <main style={{fontFamily:'system-ui',maxWidth:1000,margin:'32px
auto',padding:20}}><h1>{title}</h1><p>Independent field-based developer inspection tool. Synthetic
data only.</p><textarea value={input} onChange={e=>setInput(e.target.value)}
style={{width:'100%',height:100}}/><p><button onClick={submit}>Process
request</button></p>{error&&<p role="alert">{error}</p>}{result&&<><section
style={{height:180,border:'1px solid #ddd'}}><ReactFlow nodes={nodes} edges={edges}
fitView><Background/><Controls/></ReactFlow></section><h2>Inspection result</h2><pre style={{whiteSp
ace:'pre-wrap',background:'#f4f5f7',padding:18}}>{JSON.stringify(result,null,2)}</pre></>}</main>
}
createRoot(document.getElementById('root')!).render(<App/>);
```

20. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/frontend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "lib": [
      "ES2020",
      "DOM"
    ]
  },
  "include": [
    "src.tsx"
  ]
}
```

21. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/frontend/vite.config.ts

```
import { defineConfig } from 'vite';
export default defineConfig({server:{port:9012,proxy:{'/api':'http://localhost:8012'}}});
```

22. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/requirements.txt

```
fastapi>=0.110,<1
uvicorn>=0.29,<1
pydantic>=2.6,<3
pytest>=8,<10
httpx>=0.27,<1
```

23. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/02_recursive_feedback_engine/tests/test_api.py

```
from fastapi.testclient import TestClient
from backend.main import app
client=TestClient(app)
def test_feedback():
 d=client.post('/api/decisions',json={'request':'Quote freight','carrier':'Carrier A'}).json()
 f=client.post('/api/feedback',json={'decision_id':d['decision_id'],'corrected_carrier':'Carrier
B'}).json()
 c=client.post('/api/confirm',json={'correction_id':f['correction_id'],'confirmed':True});assert
c.status_code==200
 comparison=client.post('/api/compare',json={'request':'Quote freight again'}).json()
 assert comparison['with_feedback']['carrier']=='Carrier B'
```

24. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY shared /app
COPY 03_understanding_gap_detector/backend /app/backend
COPY 03_understanding_gap_detector/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
ENV PYTHONPATH=/app
CMD ["uvicorn","backend.main:app","--host","0.0.0.0","--port","8013"]
```

25. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/README.md

```
# User Understanding Gap Detector

Independent developer tool using shared field contracts.
```

```
Backend: `PYTHONPATH=.:../shared uvicorn backend.main:app --port 8013` from this project directory
after `pip install -r requirements.txt`.
Frontend: `cd frontend && npm install && npm run dev` (port 9013).
Tests: `PYTHONPATH=.:../shared pytest -q`.
Swagger: http://localhost:8013/docs

Demo data is synthetic. SQLite DB is created on first launch.
```

26. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/backend/__init__.py

```
[EMPTY FILE]
```

27. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/backend/main.py

```
from fastapi import FastAPI,HTTPException
from pydantic import BaseModel,Field
from fieldcore import uid
from store import Store
import os
app=FastAPI(title='Understanding Gap Detector')
store=Store(os.getenv('DB_PATH','gaps.sqlite'))
class Request(BaseModel):
 request:str=Field(min_length=1)
 available:dict[str,str]=Field(default_factory=dict)
class Resolution(BaseModel):
 analysis_id:str
 attribute:str
 value:str=Field(min_length=1)
# Graph edges express prerequisites. Multiple options can be modeled by alternative sets.
REQUIRES={'choose_cheaper_carrier':[{'baseline_shipment','rate_options','delivery_limit'}],
'baseline_shipment':[{'shipment_reference'}], 'rate_options':[{'origin','destination'}],
'delivery_limit':[]}
def minimal_missing(available:set[str],target='choose_cheaper_carrier'):
 def walk(node,stack):
  if node in available:return [set()]
  if node in stack:raise ValueError('dependency cycle')
  alternatives=REQUIRES.get(node,[])
  if not alternatives:return [{node}]
  results=[]
  for group in alternatives:
   choices=[set()]
   for child in sorted(group):
    sub=walk(child,stack|{node})
    choices=[a|b for a in choices for b in sub]
   results+=choices
  return results
 options=walk(target,set());min_size=min(map(len,options));return sorted([sorted(x) for x in options
 if len(x)==min_size])[0]
@app.post('/api/gaps/analyze')
def analyze(req:Request):
 missing=minimal_missing(set(req.available))
 x={'analysis_id':uid(),'request':req.request,'available':req.available,'missing':missing,'state':'U
NRESOLVED' if missing else 'CONFIRMED','clarifications':[{'attribute':k,'question':f'Please provide
{k.replace("_"," ")}.'} for k in missing]}
 store.save('analysis',x,x['analysis_id']);store.event('GAP_ANALYZED',x['analysis_id'],x);return x
@app.post('/api/gaps/resolve')
def resolve(r:Resolution):
 x=store.get(r.analysis_id)
 if not x or 'analysis_id' not in x:raise HTTPException(404,'analysis not found')
 updated={**x['available'],r.attribute:r.value};missing=minimal_missing(set(updated))
 result={'analysis_id':uid(),'parent_analysis_id':r.analysis_id,'request':x['request'],'available':u
pdated,'missing':missing,'state':'UNRESOLVED' if missing else
'CONFIRMED','clarifications':[{'attribute':k,'question':f'Please provide {k.replace("_"," ")}.'} for
 k in missing]}
 store.save('analysis',result,result['analysis_id']);store.event('GAP_RESOLVED',result['analysis_id'
],result);return result
@app.get('/api/gaps/{analysis_id}')
def get(analysis_id:str):
 x=store.get(analysis_id)
 if not x:raise HTTPException(404,'analysis not found')
 return x
```

28. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/frontend/index.html

```
<html><head><meta name="viewport" content="width=device-width,initial-scale=1" /></head><body><div
id="root"></div><script type="module" src="/src.tsx"></script></body></html>
```

29. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/frontend/package.json

```
{
  "name": "03_understanding_gap_detector",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "tsc && vite build",
    "test": "vitest run"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "reactflow": "^11.11.4"
  },
  "devDependencies": {
    "typescript": "^5.6.3",
    "vite": "^5.4.10",
    "vitest": "^2.1.4",
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1"
  }
}
```

30. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/frontend/src.tsx

```
import React, {useState} from 'react';
import {createRoot} from 'react-dom/client';
import ReactFlow, {Background,Controls} from 'reactflow';
import 'reactflow/dist/style.css';
const title="User Understanding Gap Detector";
function App() {
 const [input,setInput]=useState('Same shipment as last week, but two days earlier.');
 const [result,setResult]=useState<any>(null); const [error,setError]=useState('');
 const endpoint="/api/gaps/analyze";
 async function submit() {try{const body=endpoint==='/api/decisions'?{request:input,carrier:'Carrier
 A'}:{request:input};const r=await fetch(endpoint,{method:'POST',headers:{'Content-Type':'applicatio
n/json'},body:JSON.stringify(body)});const x=await r.json();if(!r.ok)throw
Error(JSON.stringify(x));setResult(x);setError('')}catch(e){setError(String(e))}}
 const nodes=result?[{id:'1',position:{x:0,y:20},data:{label:'User
request'}},{id:'2',position:{x:240,y:20},data:{label:'Context
field'}},{id:'3',position:{x:480,y:20},data:{label:'Developer inspection'}}]:[];
 const edges=[{id:'e1',source:'1',target:'2'},{id:'e2',source:'2',target:'3'}];
 return <main style={{fontFamily:'system-ui',maxWidth:1000,margin:'32px
auto',padding:20}}><h1>{title}</h1><p>Independent field-based developer inspection tool. Synthetic
data only.</p><textarea value={input} onChange={e=>setInput(e.target.value)}
style={{width:'100%',height:100}}/><p><button onClick={submit}>Process
request</button></p>{error&&<p role="alert">{error}</p>}{result&&<><section
style={{height:180,border:'1px solid #ddd'}}><ReactFlow nodes={nodes} edges={edges}
fitView><Background/><Controls/></ReactFlow></section><h2>Inspection result</h2><pre style={{whiteSp
ace:'pre-wrap',background:'#f4f5f7',padding:18}}>{JSON.stringify(result,null,2)}</pre></>}</main>
}
createRoot(document.getElementById('root')!).render(<App/>);
```

31. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/frontend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
```

```
    "esModuleInterop": true,
    "lib": [
      "ES2020",
      "DOM"
    ]
  },
  "include": [
    "src.tsx"
  ]
}
```

32. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/frontend/vite.config.ts

```
import { defineConfig } from 'vite';
export default defineConfig({server:{port:9013,proxy:{'/api':'http://localhost:8013'}}});
```

33. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/requirements.txt

```
fastapi>=0.110,<1
uvicorn>=0.29,<1
pydantic>=2.6,<3
pytest>=8,<10
httpx>=0.27,<1
```

34. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/03_understanding_gap_detector/tests/test_api.py

```
from fastapi.testclient import TestClient
from backend.main import app,minimal_missing
client=TestClient(app)
def test_gaps():
 r=client.post('/api/gaps/analyze',json={'request':'Ship as usual but cheaper'});assert
r.status_code==200
 assert 'shipment_reference' in r.json()['missing']
 assert minimal_missing({'shipment_reference','origin','destination','delivery_limit'})==[] or minim
al_missing({'shipment_reference','origin','destination','delivery_limit'})==['choose_cheaper_carrier
']
```

35. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/README.md

```
# nemox RICO-7 Three Field-Based Developer Tools

Independent reference implementations for a freight-forwarding hackathon. No affiliation or access
to nemox internal systems is claimed.

## Run

From repository root: `docker compose up --build`

API documentation: http://localhost:8011/docs, http://localhost:8012/docs,
http://localhost:8013/docs.

Run frontends independently: `cd <solution>/frontend && npm install && npm run dev`. Vite ports:
9011, 9012, 9013.

Local Python: `pip install -r <solution>/requirements.txt`; `PYTHONPATH=shared uvicorn
backend.main:app --port <solution_port>` from each solution directory.

Tests: `cd <solution> && PYTHONPATH=.:../shared pytest -q`.

## Demonstrations

A: POST /api/analyze with `{"request":"Same shipment as last week, but two days earlier."}`.
B: POST /api/decisions, POST /api/feedback, POST /api/confirm, POST /api/compare.
C: POST /api/gaps/analyze, POST /api/gaps/resolve.

## Boundaries

These are functional prototype implementations with synthetic data. They do not include production
identity management, event-stream concurrency, operational TMS integration, or a real LLM provider.
React interfaces are lightweight inspection shells.
```

36. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/docker-compose.yml

```
services:
  intent:
    build: {context: ., dockerfile: 01_user_intent_inspector/Dockerfile}
    ports: ['8011:8011']
  feedback:
    build: {context: ., dockerfile: 02_recursive_feedback_engine/Dockerfile}
    ports: ['8012:8012']
  gaps:
    build: {context: ., dockerfile: 03_understanding_gap_detector/Dockerfile}
    ports: ['8013:8013']
```

37. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/shared/__init__.py

```
[EMPTY FILE]
```

38. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/shared/fieldcore.py

```
from datetime import datetime, timezone
from uuid import uuid4
from typing import Literal
from pydantic import BaseModel, Field

def now(): return datetime.now(timezone.utc).isoformat()
def uid(): return str(uuid4())
class SourceRef(BaseModel):
    source_id: str
    segment: str = ''
class FieldRecord(BaseModel):
    field_id: str = Field(default_factory=uid)
    field_type: str
    scope_id: str
    version: int = 1
    state: Literal['INITIAL','ENRICHED','UNRESOLVED','CORRECTED','CONFIRMED','SUPERSEDED']='INITIAL'
    values: dict = Field(default_factory=dict)
    sources: list[SourceRef] = Field(default_factory=list)
class Relation(BaseModel):
    relation_id: str = Field(default_factory=uid)
    source_id: str
    predicate: str
    target_id: str
    source_ref: str
    status: Literal['proposed','confirmed','superseded']='proposed'
class Event(BaseModel):
    event_id: str = Field(default_factory=uid)
    kind: str
    aggregate_id: str
    correlation_id: str
    payload: dict
    occurred_at: str = Field(default_factory=now)
TRANSITIONS={
 'INITIAL':{'ENRICHED','UNRESOLVED'},
 'ENRICHED':{'UNRESOLVED','CONFIRMED','CORRECTED'},
 'UNRESOLVED':{'ENRICHED','CORRECTED'},
 'CORRECTED':{'CONFIRMED','UNRESOLVED'},
 'CONFIRMED':{'SUPERSEDED'},
 'SUPERSEDED':set(),
}
def transition(record:FieldRecord, target:str):
    if target not in TRANSITIONS[record.state]: raise ValueError(f'invalid transition {record.state}
 -> {target}')
    return record.model_copy(update={'state':target,'version':record.version+1})
```

39. NEMOX_AI_RICO7_THREE_SOLUTIONS_IMPLEMENTATION/shared/store.py

```
import sqlite3, json
from pathlib import Path
from fieldcore import uid,now
class Store:
 def __init__(self,path):
  self.path=path; self.db=sqlite3.connect(path,check_same_thread=False)
  self.db.execute('CREATE TABLE IF NOT EXISTS records (id TEXT PRIMARY KEY, kind TEXT NOT NULL,
```

```
payload TEXT NOT NULL, created_at TEXT NOT NULL)')
  self.db.execute('CREATE TABLE IF NOT EXISTS events (event_id TEXT PRIMARY KEY, correlation_id TEXT
 NOT NULL, kind TEXT NOT NULL, payload TEXT NOT NULL, created_at TEXT NOT NULL)')
  self.db.commit()
 def save(self,kind,obj,id=None):
  id=id or uid(); self.db.execute('INSERT INTO records VALUES
(?,?,?,?)',(id,kind,json.dumps(obj),now()));self.db.commit();return id
 def get(self,id):
  row=self.db.execute('SELECT payload FROM records WHERE id=?',(id,)).fetchone();return
json.loads(row[0]) if row else None
 def list(self,kind):
  return [{'id':r[0],**json.loads(r[1])} for r in self.db.execute('SELECT id,payload FROM records
WHERE kind=? ORDER BY created_at',(kind,))]
 def event(self,kind,correlation_id,payload):
  eid=uid();self.db.execute('INSERT INTO events VALUES
(?,?,?,?,?)',(eid,correlation_id,kind,json.dumps(payload),now()));self.db.commit();return eid
```